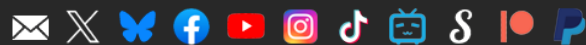


Inigo Quilez :: articles :: sphere density - 2015



Motivation

Sometimes you need to do some pixel color processing selectively in some areas of your CG image. Usually this can be done in two dimensions, or more generally, through lighting. Indeed, while effects studios doing live action and photoreal rendering treat lighting as a way achieve realism and believability, animation studios see lighting as a way to paint colors in the screen in a way that lets them realize their artistic vision. Point-lights, rods, and volumes become brushes really. However, many of these light-based coloring techniques lack thickness or a sense of participating media - they only affect surfaces but not the space between them. In this article we'll see how to develop one volumetric tool that you can use to implement such colorization techniques, or to implement simple traditional fog, but localized in space.



Analytically integrated volumetric media (no raymarching)

The idea is that we want to place a sphere in some location in space and accumulate some participating media density along the view ray within the volume. Then use that accumulation to drive some visual effect (as simple as fog or as complex as image distortion, color grading, texturing or some procedural effect. But it all starts from accumulating density of media along a segment or space. Which can be easily implemented with volumetric raymarching. However this is slow. In this article we are going to see how to achieve this analytically without raymarching, by using only a few maths operations.

We used this technique in the VR film "[Henry](#)", and I know it was used as well in Epic's game "Robo Recall".

The code

The first step is to detect if the current pixel overlaps with the spherical volume, and early skip if it doesn't. We can do this by a simple ray-sphere intersection test. If an intersection happens, then we are interested in knowing the entry and exit point of the ray with the sphere, so that we can later integrate the fog amount along the segment of the ray which is inside the sphere. The raytrace is very simple, and we describe it here in the following paragraph only for reference:

For points in space $\mathbf{x}$, a sphere with center $\mathbf{sc}$ and radius r is defined by

$$|\mathbf{x} - \mathbf{sc}| = r$$

and a ray originating at the camera position $\mathbf{ro}$ and going through our pixel in direction $\mathbf{rd}$ can be defined as

$$\mathbf{x} = \mathbf{ro} + t \cdot \mathbf{rd}$$

with $t > 0$ for rays shooting forward and $|\mathbf{rd}| = 1$ for an isotropic metric. The overlap and intersection of the pixel ray with the sphere can be solved by replacing $\mathbf{x}$ in the sphere equation with the ray equation:

$$|\mathbf{ro} + t \cdot \mathbf{rd} - \mathbf{sc}| = r$$

Since $(a+b)^2 = a^2 + b^2 + 2ab$, squaring both sides of the equation and expanding the result gives us

$$t^2 + t \cdot 2\langle \mathbf{rd}, \mathbf{ro} - \mathbf{sc} \rangle + |\mathbf{ro} - \mathbf{sc}|^2 - r^2 = 0$$

which is a quadratic in t with solution $t = -b \pm \sqrt{b^2 - c}$ with $b = \langle \mathbf{rd}, \mathbf{ro} - \mathbf{sc} \rangle$ and $c = |\mathbf{ro} - \mathbf{sc}|^2 - r^2$ as long as $b^2 > c$ of course.

Once we have the entry and exit points parametrized by t , we are almost ready for integrating the fog. Before that we just have to account for the case the sphere is completely behind the camera ($t_2 < 0.0$) or completely hidden behind the depthbuffer ($t_1 > \mathbf{dbuffer}$). Then we have to clip the segment so that we only integrate from the camera position forward and no further than indicated by the depth buffer, which we can do by performing:

$$T_1 = \max(-b-h, 0)$$

$$T_2 = \min(-b+h, \text{depthbuffer})$$

We can now integrate the fog along the segment. We can choose from a variety of fog density functions. We chose one that peaks at the center of the sphere where the density is maximum (1.0) and decays quadratically until it reaches zero at the surface of the sphere.

$$D(x) = 1 - \frac{|x - sc|^2}{r^2}$$

This function is easily integrable analytically:

$$F = \int_{T_1}^{T_2} D(t) dt = \int_{T_1}^{T_2} \frac{r^2 - |ro - sc + t \cdot rd|^2}{r^2} dt = \frac{-1}{r^2} \int_{T_1}^{T_2} (c + 2bt + t^2) dt = \frac{-1}{r^2} \left[ct + bt^2 + \frac{t^3}{3} \right]_{T_1}^{T_2}$$

It might be convenient now to normalize the accumulated fog F such that it takes the value 1 in the extreme case of the ray going right through its center all the way from its surface to the back side. In that geometric configuration we have $\mathbf{c}=\mathbf{0}$ and $\mathbf{b}=-\mathbf{r}$, so

$$F_{\max} = \{c = 0, b = -r\} = \frac{4}{3}r$$

Therefore the final expression, ready for implementation, is

$$NF = F / F_{\max} = \frac{-3}{4r^3} \left[ct + bt^2 + \frac{t^3}{3} \right]_{T_1}^{T_2}$$

As a curiosity, note that when the sphere does not overlap with the camera or the scene, then

$$NF_{full} = \{T_2 - T_1 = t_2 - t_1 = 2h\} = \frac{-3}{4r^3} \left[ct + bt^2 + \frac{t^3}{3} \right]_{-h}^{+h} = \left(\frac{h}{r} \right)^3$$

These results are almost ready for being implemented in code. Before that, it is worth noting that some floating point precision can be gained by recasting the whole problem into the unit sphere (centered at the origin and with radius 1), in which case the final implementation is:

```
float computeFog( vec3 ro, vec3 rd,    // ray origin, ray direction
                 vec3 sc, float sr,    // sphere center, sphere radius
                 float dbuffer )
{
    // normalize the problem to the canonical sphere
    float ndbuffer = dbuffer / sr;
    vec3 rc = (ro - sc)/sr;

    // find intersection with sphere
    float b = dot(rd,rc);
    float c = dot(rc,rc) - 1.0f;
    float h = b*b - c;

    // not intersecting
    if( h<0.0f ) return 0.0f;
    h=sqrtf( h );
    float t1=-b - h;
    float t2=-b + h;

    // not visible (behind camera or behind ndbuffer)
    if( t2<0.0f || t1>ndbuffer ) return 0.0f;

    // clip integration segment from camera to ndbuffer
    t1 = max( t1, 0.0f );
    t2 = min( t2, ndbuffer );

    // analytical integration of an inverse squared density
    float i1 = -(c*t1 + b*t1*t1 + t1*t1*t1/3.0f);
    float i2 = -(c*t2 + b*t2*t2 + t2*t2*t2/3.0f);
    return (i2-i1)*(3.0f/4.0f);
}
```

Here goes the code above running live: <https://www.shadertoy.com/view/XljGDy>

inigo quilez - learning computer graphics since 1994